

LAPORAN PRAKTIKUM
PRAKTIKUM PENGEMBANGAN GAME
PERTEMUAN 7
ASSET FAB DAN WIDGET BLUEPRINT (UI)



Disusun oleh:

Nama : Hafidz Rizqullah Prasetya
NIM : 24/535493/SV/24243
NIU : 535493
Kelas : PL5A1
Dosen Pengampu : Yusron Fuadi, S.Sn., M.Sn.
Asisten : Ezra Bariq Rizqullah / Ikhwan Hanif Firdaus

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Dasar Teori	4
2.1 Ekosistem Fab dan Distribusi Aset Digital Terpadu	4
2.2 Arsitektur Material Landscape Multi-Layer dan Layer Info	4
2.3 Distribusi Vegetasi Lingkungan Melalui Foliage System	5
2.4 Arsitektur User Interface Menggunakan UMG dan Animasi Widget	5
2.5 Pemisahan Ketergantungan via Blueprint Interface dan Game Instance	6
Hasil dan Pembahasan	8
3.1 Dokumentasi Praktikum	8
3.1.1 Eksplorasi FAB Marketplace dan Integrasi Aset Landscape Pro 2.0	8
3.1.2 Implementasi Material Landscape dan Pengecatan Lapisan Permukaan	10
3.1.3 Penerapan Vegetasi Foliage Manual dan Foliage Mode	11
3.1.4 Perancangan Widget Blueprint dan Animasi UI	12
3.1.5 Pembuatan dan Konfigurasi Blueprint Interface	14
3.1.6 Integrasi Logika UI dengan Gameplay dan Komponen Kesehatan	15
3.2 Hasil Akhir	18
Kesimpulan	19
Daftar Pustaka	20

Tujuan Praktikum

1. Mahasiswa mampu melakukan eksplorasi, pengunduhan, serta integrasi aset lingkungan tiga dimensi dari marketplace terpadu Fab ke dalam proyek Unreal Engine 5.8 [1, 2].
2. Mahasiswa mampu mengonfigurasi dan mengaplikasikan material bentang alam multi-layer (*Landscape Material*) menggunakan kontainer data *Layer Info* untuk kebutuhan pengecatan tekstur permukaan lingkungan [1, 4].
3. Mahasiswa mampu menerapkan aset vegetasi lingkungan (*foliage*) secara presisi melalui metode penempatan manual maupun secara prosedural memanfaatkan *Foliage Mode* [1, 5].
4. Mahasiswa mampu merancang antarmuka visual pemain (*User Interface*) berbasis UMG (*Unreal Motion Graphics*), mencakup penataan hierarki kanvas, batang status kesehatan (*HealthBar*), serta animasi respons visual dinamis [1, 6, 7].
5. Mahasiswa mampu mengintegrasikan sistem antarmuka ke dalam siklus hidup permainan secara modular melalui implementasi *Blueprint Interface* dan *Game Instance*, menghubungkan penurunan nilai darah pemain saat menerima luka langsung ke indikator layar [1, 8, 9].

Dasar Teori

2.1 Ekosistem Fab dan Distribusi Aset Digital Terpadu

Industri pengembangan game modern sangat bergantung pada ketersediaan pustaka aset digital berkualitas tinggi guna mempercepat fase prototipe dan produksi [2]. Epic Games meluncurkan platform Fab sebagai pasar digital terpadu yang menggabungkan berbagai ekosistem aset terdahulu, seperti Unreal Engine Marketplace, Sketchfab, ArtStation Marketplace, dan Quixel [3]. Penyatuan ini menghadirkan repositori aset tiga dimensi agnostik yang dapat dimanfaatkan lintas perangkat lunak kreasi konten, mulai dari Unreal Engine, Unity, Blender, hingga Maya [2].

Dalam konteks kompatibilitas versi, paket aset lingkungan seperti *Landscape Pro 2.0* umumnya dirilis untuk versi engine tertentu (misalnya UE 5.5). Unreal Engine menyediakan mekanisme impor adaptif yang memungkinkan pengembang memuat aset dari versi terdahulu ke dalam engine mutakhir (UE 5.8) [1]. Melalui jendela konfigurasi Library, aset dikonversi secara transparan ke format uasset lokal, memastikan shader, material instanced, dan hierarki static mesh tetap utuh tanpa merusak integritas berkas proyek [1].

2.2 Arsitektur Material Landscape Multi-Layer dan Layer Info

Bentang alam pada Unreal Engine membutuhkan penanganan material khusus yang mampu membaurkan berbagai jenis tekstur permukaan (seperti tanah, rumput, lumpur, dan bebatuan) pada koordinat medan yang sama [4]. Sistem ini diimplementasikan melalui material berlapis (*Landscape Material*) yang memanfaatkan ekspresi Landscape LayerBlend [4].

$$\mathbf{C}_{\text{final}}(u, v) = \sum_{i=1}^N w_i(u, v) \cdot \mathbf{C}_i(u, v) \quad (1)$$

Persamaan di atas merepresentasikan kalkulasi warna permukaan akhir $\mathbf{C}_{\text{final}}$ pada titik koordinat UV (u, v) , di mana $\mathbf{C}_i$ merupakan nilai warna difus dari layer material ke- i , dan w_i adalah faktor bobot (*weight*) sapuan kuas yang ternormalisasi ($\sum w_i = 1$). Agar

sistem rendering mampu menyimpan dan mengevaluasi bobot sapuan cat pada setiap titik verteks secara persisten, Unreal Engine memerlukan kontainer data yang disebut *Layer Info* [1]. Objek aset ini menyimpan parameter fisik permukaan (seperti respon jejak kaki dan gesekan fisika) serta memori alokasi bobot blending, sehingga lapisan cat yang berbeda dapat dipadukan secara mulus di atas satu mesh landscape yang sama [4].

2.3 Distribusi Vegetasi Lingkungan Melalui Foliage System

Vegetasi lingkungan (*foliage*) mencakup pepohonan, semak-semak, dan rerumputan yang memberikan kepadatan visual pada dunia game terbuka [5]. Unreal Engine menyediakan dua metodologi penempatan foliage:

1. **Penempatan Manual (*Manual Placement*):** Penempatan static mesh secara langsung melalui proses seret dan lepas (*drag-and-drop*) dari Content Drawer ke level. Metode ini memberikan kontrol tata letak presisi tinggi, umumnya diterapkan pada pohon-pohon signifikan (*hero assets*) atau penanda navigasi visual [1].
2. **Foliage Mode Painting:** Pendekatan instansiasi prosedural di mana sekumpulan *Static Mesh Foliage* didaftarkan ke panel Foliage. Pengembang dapat mengecat vegetasi secara masif dengan parameter kerapatan (*density*), skala acak (*scaling*), orientasi rotasi acak, dan penyesuaian terhadap kemiringan normal permukaan tanah [5]. Teknik ini mengelompokkan ribuan model ke dalam satu *Hierarchical Instanced Static Mesh* (HISM) guna meminimalkan beban panggilan penggambaran grafis (*draw calls*) [5].

2.4 Arsitektur User Interface Menggunakan UMG dan Animasi Widget

Sistem antarmuka pengguna pada Unreal Engine dibangun menggunakan kerangka kerja *Unreal Motion Graphics* (UMG) yang berbasis slate [6]. Setiap antarmuka dienkapsulasi ke dalam *Widget Blueprint*, yang memiliki dua mode kerja utama: mode *Designer* untuk tata letak visual berbasis kanvas dan mode *Graph* untuk pemrograman logika interaksi pemain [6].

Komponen Canvas Panel bertindak sebagai kontainer induk berorientasi koordinat absolut yang mendukung sistem penjangkaran (*anchors*). Penjangkaran memastikan elemen UI (seperti indikator status di pojok kiri atas) tetap berada pada posisi proporsional terhadap berbagai rasio aspek monitor pemain [6]. Di dalam kanvas tersebut, komponen visual seperti Progress Bar digunakan untuk menampilkan rasio metrik numerik kontinu (darah, stamina, energi). Nilai isian bar diatur melalui properti persentase:

$$\text{Percent} = \text{clamp} \left(\frac{\text{CurrentHealth}}{\text{MaxHealth}}, 0.0, 1.0 \right) \quad (2)$$

Guna memperkaya umpan balik sensorik pemain (*visual feedback*), UMG menyediakan panel editor linimasa *Widget Animation* [7]. Pengembang dapat memanipulasi sub-jalur *Transform (Translation X, Y)* untuk menghasilkan efek getaran cepat (*shake/jiggle*) saat nilai darah mengalami perubahan, memperjelas persepsi bahaya saat karakter menerima kerusakan [1].

2.5 Pemisahan Ketergantungan via Blueprint Interface dan Game Instance

Dalam perancangan perangkat lunak game berorientasi objek, ketergantungan langsung antar-kelas (*hard-casting*) merupakan pola anti-desain yang meningkatkan beban komputasi dan memicu kesalahan sirkular [8]. Unreal Engine mengatasi kendala ini melalui pemanfaatan *Blueprint Interface* (BPI). BPI mendefinisikan kontrak fungsi tanpa implementasi konkret, memungkinkan komunikasi antar-aktor tanpa memerlukan referensi kelas spesifik [8].

Tabel 1: Perbandingan komunikasi antar-Blueprint menggunakan Hard Casting versus Blueprint Interface [8, 1].

Parameter Evaluasi	Hard Casting (Cast To)	Blueprint Interface (BPI)
Tingkat Ketergantungan	Sangat ketat (<i>tightly coupled</i>); memuat referensi kelas ke memori.	Terisolasi penuh (<i>loosely coupled</i>); referensi kelas tidak dimuat.
Penanganan Kegagalan	Memicu percabangan <i>Cast Failed</i> jika target objek berbeda kelas.	Diabaikan secara aman tanpa pesan galat jika objek tidak mengimplementasikan interface.
Efisiensi Kompilasi	Perubahan pada kelas target memaksa rekompilasi seluruh pemanggil.	Perubahan internal kelas tidak memengaruhi penyeru pesan interface.
Aksesibilitas Objek	Terbatas pada hierarki pewarisan kelas yang sama atau subkelasnya.	Bersifat universal; dapat dipanggil ke sembarang aktor atau widget.

Untuk mengelola referensi widget antarmuka secara global sepanjang sesi aplikasi, sistem memanfaatkan *Game Instance* [9]. Berbeda dengan aktor level atau Player Controller

yang dapat dihancurkan saat perpindahan level, objek Game Instance tetap persisten di memori selama proses permainan berjalan. Menyimpan referensi widget di Game Instance via antarmuka memastikan sistem luar (seperti komponen karakter yang menerima damage) dapat memperbarui tampilan HUD secara instan tanpa perlu melacak hierarki viewport secara manual [1, 9].

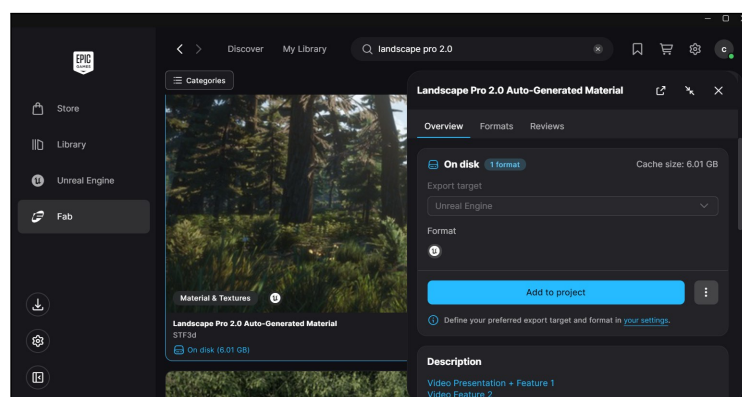
Hasil dan Pembahasan

Praktikum Pertemuan 7 dilaksanakan menggunakan Unreal Engine 5.8 pada basis proyek HAFIDZ_535493_PPG. Praktikum ini melanjutkan pengembangan prototipe permainan dari pertemuan-pertemuan sebelumnya, dengan fokus mengintegrasikan aset bentang alam Fab, memperkaya lingkungan dengan vegetasi foliage, serta membangun sistem antarmuka status kesehatan pemain (HUD) berbasis UMG yang terhubung ke logika gameplay [1].

3.1 Dokumentasi Praktikum

3.1.1 Eksplorasi FAB Marketplace dan Integrasi Aset Landscape Pro 2.0

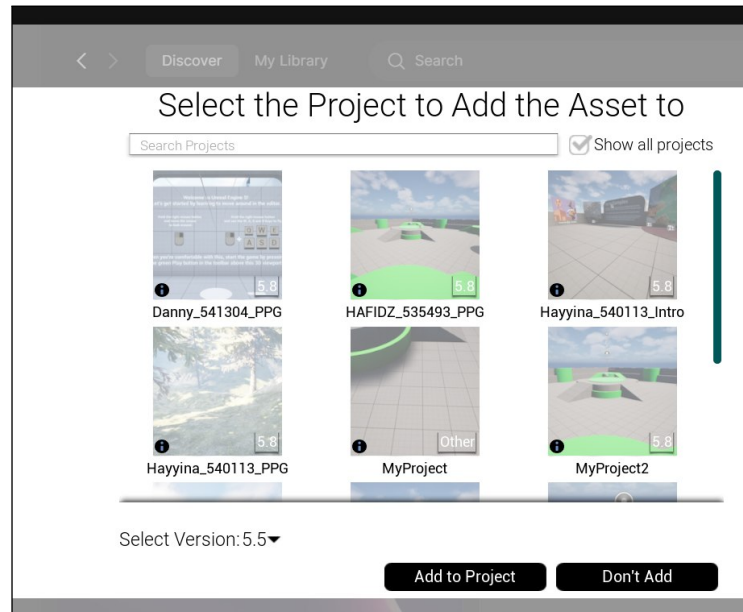
1. **Eksplorasi Aset pada Platform Fab.** Melalui aplikasi Epic Games Launcher, tab **Fab** dibuka untuk mengakses repositori aset digital terpadu. Pada bilah pencarian, kata kunci **Landscape Pro 2.0** dimasukkan untuk menemukan paket bentang alam multi-layer yang menyediakan pustaka tekstur tanah serta mesh vegetasi fotorealistik. Tampilan pencarian aset Fab diperlihatkan pada Gambar 1.



Gambar 1: Pencarian paket aset Landscape Pro 2.0 pada antarmuka marketplace digital Fab.

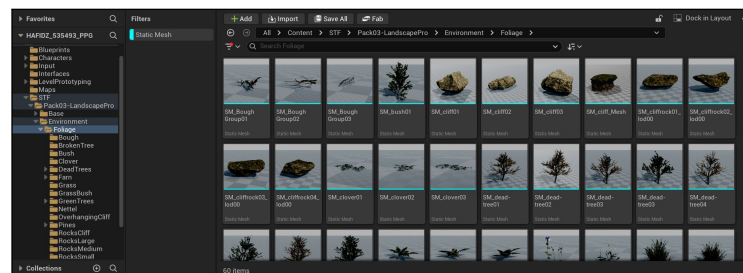
2. **Penambahan Aset ke Proyek Unreal Engine 5.8.** Paket aset *Landscape Pro 2.0* secara resmi dirancang untuk Unreal Engine 5.5. Agar dapat digunakan pada lingkungan praktikum UE 5.8, jendela dialog **Add To Project** dibuka dari Fab Library.

Opsi **Show all projects** diaktifkan, proyek praktikum HAFIDZ_535493_PPG dipilih, dan dropdown versi disetel secara eksplisit ke versi **5.5**. Proses penyesuaian versi ini ditunjukkan pada Gambar 2.



Gambar 2: Konfigurasi pengunduhan aset versi 5.5 ke dalam proyek target Unreal Engine 5.8.

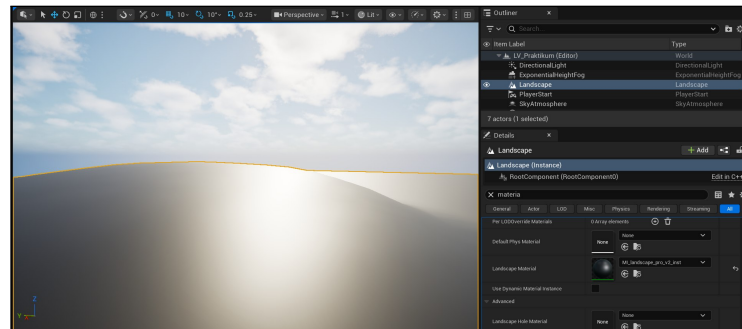
3. **Inspeksi Berkas Foliage pada Content Drawer.** Setelah proses pengunduhan selesai dan indikator ukuran tembolok (*Cache Size*) muncul, proyek dibuka kembali. Melalui Content Drawer, direktori penempatan aset diperiksa pada jalur Content/STF/Pack03-LandscapePro/Environment/Foliage. Filter jenis aset **Static Mesh** diaktifkan untuk menampilkan variasi model tiga dimensi pepohonan, ranting, dan semak belukar sebagaimana tampak pada Gambar 3.



Gambar 3: Pemeriksaan model tiga dimensi vegetasi lingkungan pada direktori paket aset Landscape Pro.

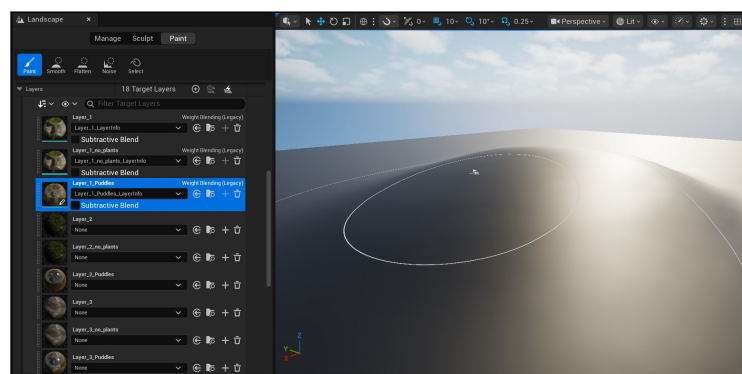
3.1.2 Implementasi Material Landscape dan Pengecatan Lapisan Permukaan

4. **Penerapan Material Multi-Layer pada Bentang Alam.** Berkas level praktikum **LV_Praktikum** dibuka. Objek Landscape yang telah dibentuk konturnya pada pertemuan sebelumnya dipilih. Melalui panel **Details**, parameter **Landscape Material** diperbarui dengan menetapkan instans material bawaan paket, yaitu **MI_landscape_pro_v2_inst**. Tampilan panel Details dan pembaruan slot material ini diperlihatkan pada Gambar 4.



Gambar 4: Penetapan material instanced MI_landscape_pro_v2_inst pada properti aktor Landscape.

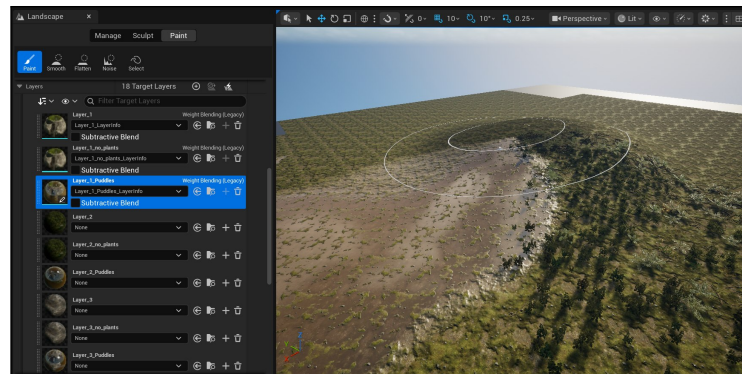
5. **Konfigurasi Layer Info pada Landscape Paint Mode.** Mode interaksi dialihkan ke **Landscape Mode** (Shift + 2) pada tab **Paint**. Tombol pengestrakan otomatis diklik untuk memunculkan daftar lapisan material yang terdefinisi di dalam shader. Pada setiap layer yang berstatus *None*, kontainer data **Layer Info** ditetapkan dari daftar opsi terbatas guna mengaktifkan alur pencatatan bobot blending tekstur. Konfigurasi panel Paint ini ditampilkan pada Gambar 5.



Gambar 5: Penetapan kontainer data Layer Info untuk setiap lapisan material pada panel Landscape Paint.

6. **Pengecatan Variasi Tekstur Bentang Alam.** Pengecatan kontur tanah dilakukan langsung pada viewport menggunakan kuas landscape. Variasi tekstur rumput (*Grass*),

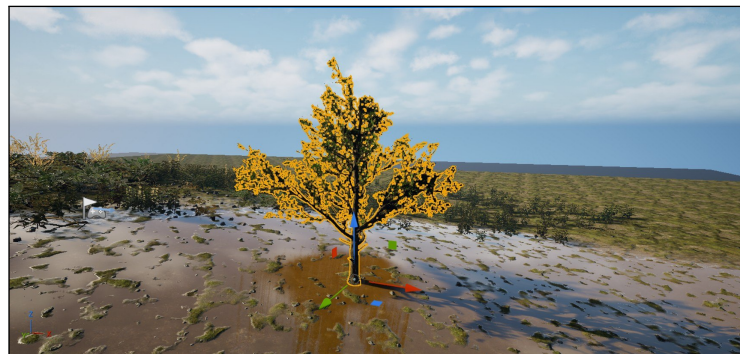
tanah gersang (*Ground*), dan genangan air (*Puddle*) disapukan secara selektif untuk menciptakan kontras visual bentang alam yang alami. Gambar 6 memperlihatkan hasil transisi gradasi tekstur permukaan yang terbentuk berkat evaluasi data bobot Layer Info.



Gambar 6: Tampilan permukaan bentang alam setelah proses pengecatan multi-layer tekstur tanah dan genangan.

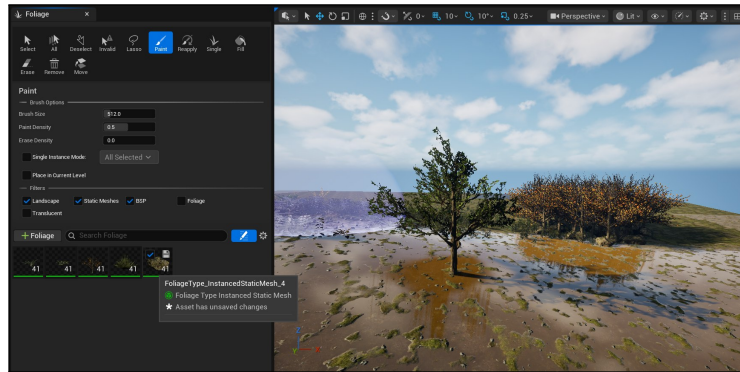
3.1.3 Penerapan Vegetasi Foliage Manual dan Foliage Mode

7. **Penempatan Vegetasi Secara Manual.** Untuk menata pepohonan tertentu pada titik strategis di sekitar area awal pemain (*Player Start*), model static mesh pohon dipilih dari Content Drawer lalu ditarik langsung ke atas permukaan tanah (*drag-and-drop*). Metode penempatan manual ini memastikan orientasi dan skala pohon heroik tertata rapi tanpa tumpang tindih acak, seperti tampak pada Gambar 7.



Gambar 7: Penempatan model pepohonan secara manual pada koordinat bentang alam melalui Content Drawer.

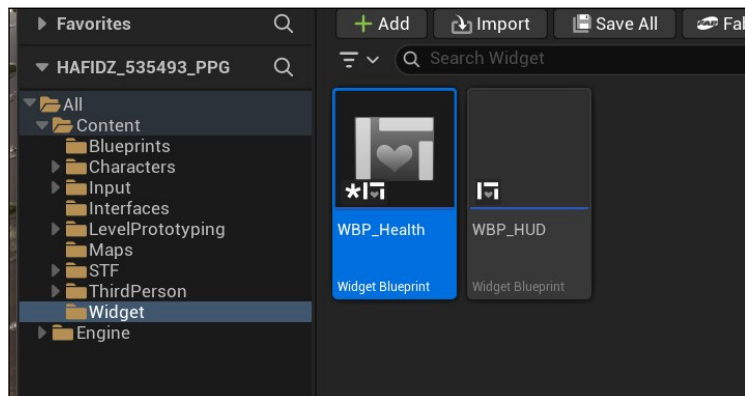
8. **Pengecatan Vegetasi Massal Menggunakan Foliage Mode.** Mode kerja dipindahkan ke **Foliage Mode** (Shift + 3). Seluruh aset bertipe *Static Mesh Foliage* pada direktori paket ditarik ke dalam kotak penampung **Drop Foliage Here**. Jenis pohon yang diinginkan dicentang, dan sapuan kuas foliage diaplikasikan di atas bentang alam untuk menghasilkan sebaran hutan yang lebat dan efisien. Gambar 8 memperlihatkan sebaran pohon hasil instansiasi prosedural tersebut.



Gambar 8: Pengecatan vegetasi massal menggunakan Foliage Mode pada area terbuka bentang alam.

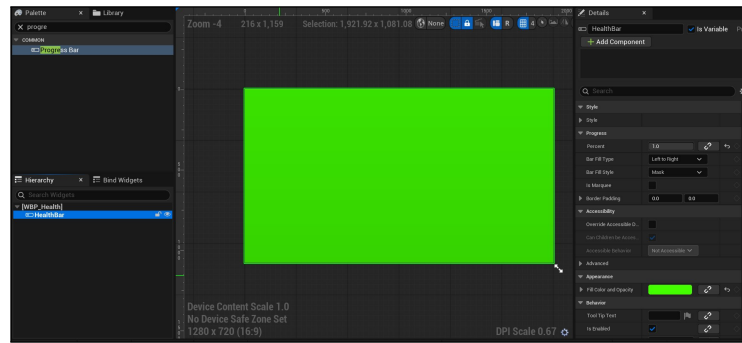
3.1.4 Perancangan Widget Blueprint dan Animasi UI

9. **Pembuatan Berkas Widget Blueprint.** Struktur direktori proyek dirapikan dengan membuat folder baru bernama **Widget** di bawah direktori **Content**. Melalui menu **Add > User Interface > Widget Blueprint**, dua berkas User Widget dibuat, yaitu **WBP_HUD** sebagai kanvas antarmuka utama dan **WBP_Health** sebagai modul penampil bar darah pemain. Susunan berkas ini diperlihatkan pada Gambar 9.



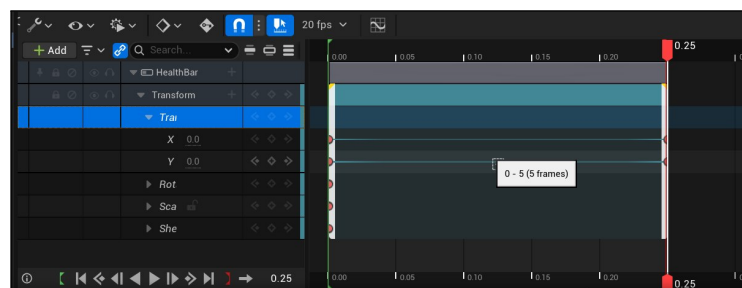
Gambar 9: Pembuatan berkas WBP_HUD dan WBP_Health di dalam direktori Content/Widget.

10. **Konfigurasi Progress Bar pada WBP_Health.** Editor WBP_Health dibuka. Komponen **Progress Bar** ditarik dari panel **Palette** ke dalam hierarki dan dinamai ulang menjadi **HealthBar**. Melalui panel **Details**, properti **Appearance > Fill Color and Opacity** diubah menjadi warna hijau terang, serta properti **Progress > Percent** disetel ke nilai **1.0** sebagai kondisi awal darah penuh. Konfigurasi ini ditampilkan pada Gambar 10.



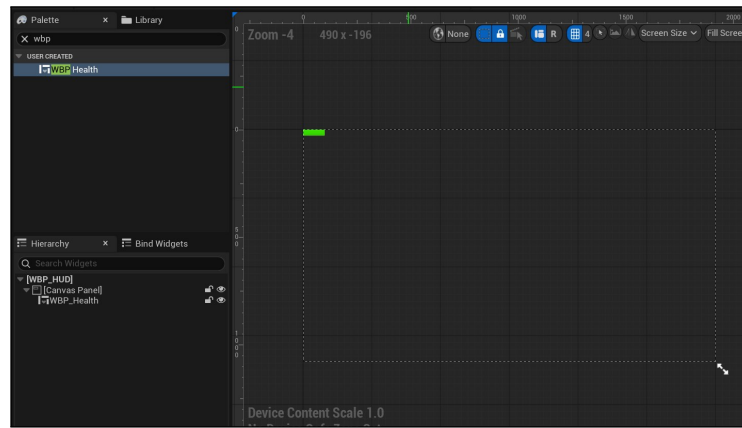
Gambar 10: Pengaturan warna batang isian dan nilai awal persentase pada komponen HealthBar.

11. **Penyusunan Animasi Getar (onUpdateHealthAnimation).** Pada panel **Animations**, aset animasi baru bernama **onUpdateHealthAnimation** ditambahkan. Komponen HealthBar didaftarkan ke linimasa dengan sub-track **Transform > Translation**. Durasi animasi diatur sepanjang **0.25 detik** pada kecepatan 20 fps, dengan beberapa keyframe translasi sumbu X dan Y acak untuk menghasilkan efek getaran visual (*shake*) saat darah berkurang, seperti tampak pada Gambar 11.



Gambar 11: Penyusunan linimasa animasi onUpdateHealthAnimation berdurasi 0.25 detik untuk efek getar UI.

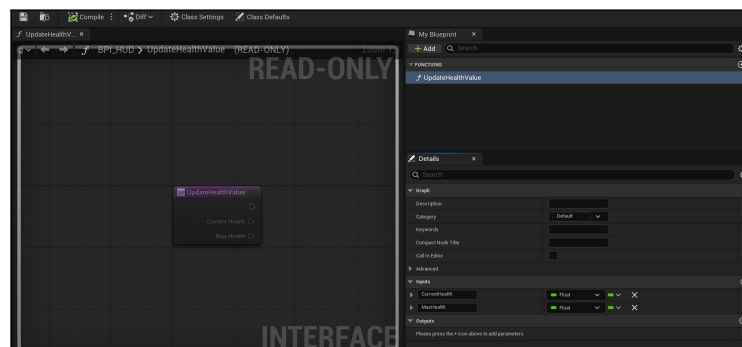
12. **Penataan Hierarki Kanvas pada WBP_HUD.** Editor WBP_HUD dibuka pada mode Designer. Komponen **Canvas Panel** ditambahkan sebagai kontainer induk. Selanjutnya, widget **WBP_Health** dimasukkan ke dalam Canvas Panel dan diposisikan pada area sudut kiri atas layar dengan margin penjangkaran yang aman dari batas tepi monitor. Penataan hierarki kanvas visual ini diperlihatkan pada Gambar 12.



Gambar 12: Penataan letak komponen modular WBP_Health di dalam Canvas Panel WBP_HUD pada area kiri atas layar.

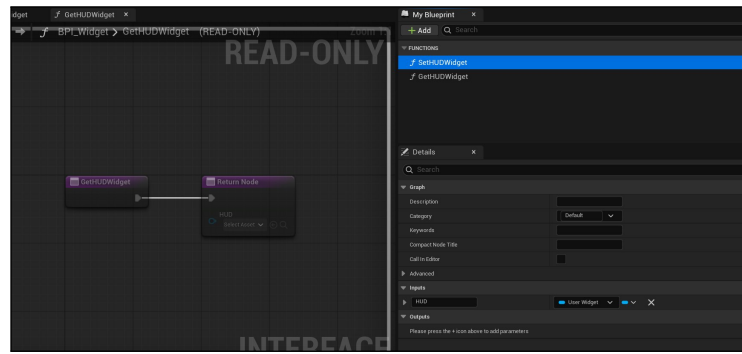
3.1.5 Pembuatan dan Konfigurasi Blueprint Interface

13. **Pembuatan Antarmuka BPI_HUD.** Untuk menyediakan jalur pembaruan nilai darah tanpa ketergantungan langsung ke widget, aset *Blueprint Interface* baru dibuat dengan nama **BPI_HUD**. Di dalamnya, fungsi **UpdateHealthValue** didefinisikan dengan dua parameter input bertipe Float, yaitu **CurrentHealth** dan **MaxHealth**, tanpa parameter output. Struktur antarmuka ini tampak pada Gambar 13.



Gambar 13: Definisi fungsi UpdateHealthValue pada Blueprint Interface BPI_HUD dengan parameter input darah.

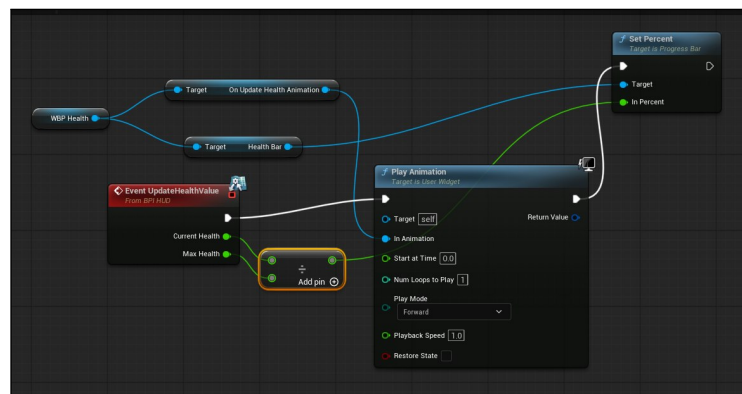
14. **Pembuatan Antarmuka BPI_Widget.** Komunikasi global antara Player Controller, Game Instance, dan karakter dikelola menggunakan antarmuka kedua, yaitu **BPI_Widget**. Dua fungsi dideklarasikan: fungsi **SetHUDWidget** dengan input referensi User Widget (**HUD**) untuk menyimpan objek widget, serta fungsi **GetHUDWidget** dengan output referensi User Widget (**HUD**) untuk mengambil objek widget aktif, seperti diperlihatkan pada Gambar 14.



Gambar 14: Deklarasi fungsi setter dan getter referensi User Widget pada antarmuka BPI_Widget.

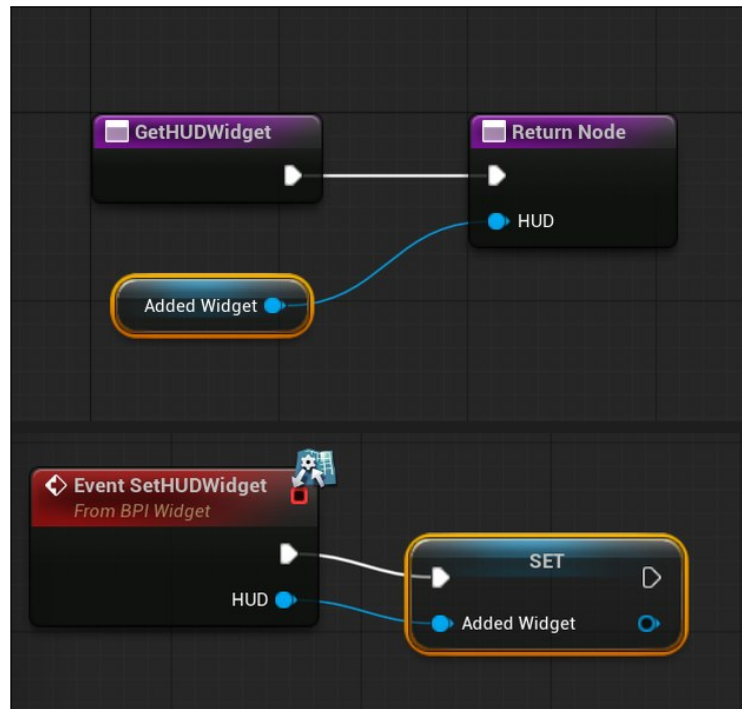
3.1.6 Integrasi Logika UI dengan Gameplay dan Komponen Kesehatan

15. **Implementasi BPI_HUD pada WBP_HUD.** Pada editor WBP_HUD, antarmuka BPI_HUD didaftarkan melalui **Class Settings**. Pada Event Graph, fungsi **Event UpdateHealthValue** diimplementasikan: garis eksekusi memanggil node **Play Animation** untuk memutar **onUpdateHealthAnimation** pada instans WBP_Health, lalu membagi **CurrentHealth** dengan **MaxHealth** untuk dihubungkan ke fungsi **Set Percent** pada komponen **HealthBar**. Rangkaian logika ini tampak pada Gambar 15.



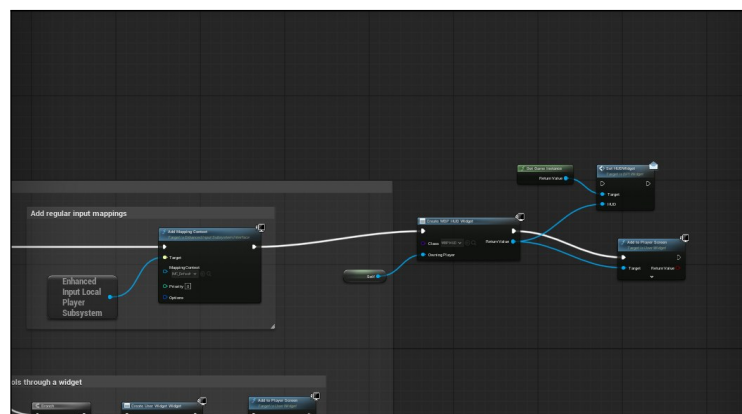
Gambar 15: Rangkaian Event Graph WBP_HUD yang memicu pemutaran animasi dan pembaruan persentase HealthBar.

16. **Penyimpanan Referensi Widget di Game Instance.** Blueprint BP_GameInstance diperbarui dengan menambahkan antarmuka **BPI_Widget** pada Class Settings serta mendeklarasikan variabel baru bernama **AddedWidget** bertipe *User Widget*. Fungsi **GetHUDWidget** disambungkan langsung untuk mengembalikan nilai variabel tersebut, sementara event **SetHUDWidget** memperbarui nilai **AddedWidget** dengan instans widget yang diterima, sebagaimana ditunjukkan pada Gambar 16.



Gambar 16: Implementasi fungsi getter dan setter antarmuka BPI_Widget pada BP_GameInstance.

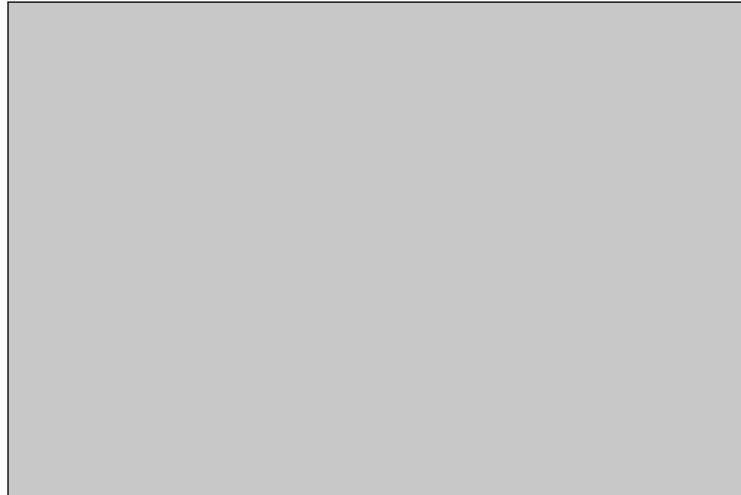
17. **Inisialisasi Pembuatan UI pada Player Controller.** Pada kelas pengontrol karakter BP_ThirdPersonPlayerController, alur inisialisasi awal disambungkan setelah penetapan konteks input. Node **Create Widget** dipanggil untuk membuat instans **WBP_HUD** dengan *Owning Player* disetel ke Self. Referensi widget yang tercipta dikirim ke Game Instance melalui fungsi pesan **Set HUDWidget (Target: BPI_Widget)**, lalu dirender ke layar pemain menggunakan node **Add to Player Screen**. Rangkaian ini diperlihatkan pada Gambar 17.



Gambar 17: Alur inisialisasi antarmuka WBP_HUD dan penampilannya ke layar pemain pada Player Controller.

18. **Pembaruan Indikator UI saat Karakter Menerima Kerusakan.** Logika Event

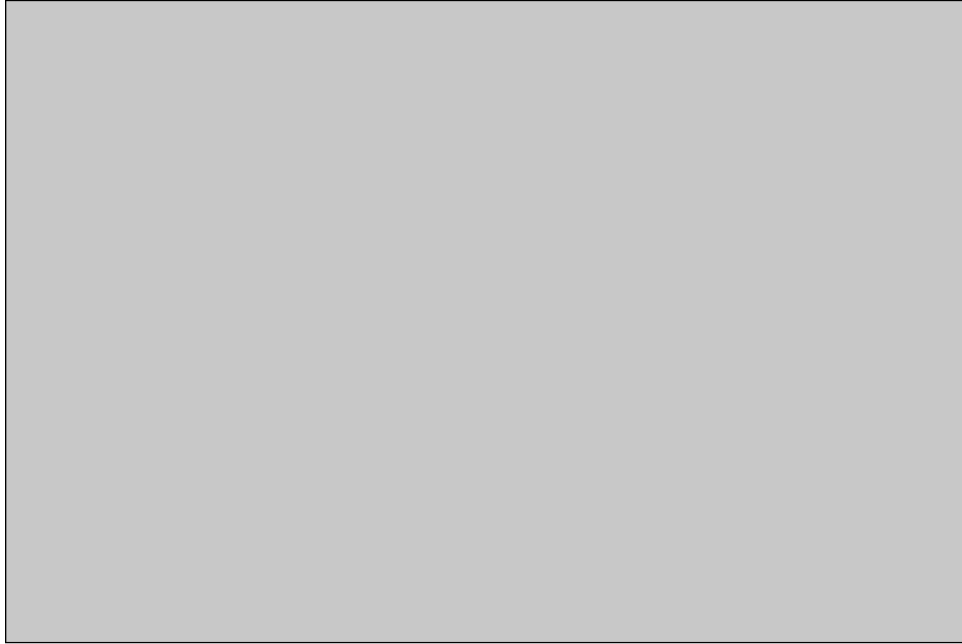
TakeDamage pada BP_ThirdPersonCharacter diperbarui. Setelah komponen kesehatan BPC_Health mengurangi nilai darah via fungsi Modify Health, alur eksekusi mengambil objek HUD aktif melalui node **Get Game Instance > Get HUDWidget (Target: BPI_Widget)**. Pesan interface **Update Health Value (Target: BPI_HUD)** kemudian dipanggil dengan memasukkan nilai CurrentHealth dan MaxHealth terbaru dari komponen kesehatan. Alur logika ini ditampilkan pada Gambar 18.



Gambar 18: Rangkaian Event Graph TakeDamage karakter yang meneruskan data kesehatan terkini ke antarmuka HUD.

3.2 Hasil Akhir

Pengujian sistem terintegrasi dilakukan langsung di dalam editor Unreal Engine 5.8 dengan membuka level **LV_Praktikum** dan menekan tombol **Play** (PIE). Saat sesi permainan dimulai, antarmuka HUD (WBP_HUD) berhasil dimunculkan secara otomatis di sudut kiri atas layar pemain dengan indikator bilah darah (HealthBar) berwarna hijau terang dalam kondisi terisi penuh (100%).



Gambar 19: Uji coba gameplay di level LV_Praktikum: indikator HealthBar hijau muncul di kiri atas, berkurang dan memainkan animasi getar saat karakter menerima damage tabrakan.

Ketika karakter digerakkan menabrak aktor pemicu bahaya (*hazard collision*), logika penanganan tabrakan memicu fungsi `Modify Health` pada `BPC_Health`. Secara instan, alur interface mengirimkan data darah terbaru menuju sistem HUD melalui Game Instance. Panjang visual batang HealthBar langsung menyusut proporsional terhadap sisa darah, dibarengi dengan pemutaran animasi getaran (*shake translation*) berdurasi 0,25 detik pada bilah antarmuka. Karakter tetap dapat bergerak secara responsif di atas bentang alam berkontur yang dihiasi oleh pepohonan foliage tanpa mengalami penurunan performa grafis, membuktikan seluruh integrasi modul praktikum telah berfungsi sempurna sesuai spesifikasi.

Kesimpulan

Praktikum Pertemuan 7 memberikan pemahaman komprehensif mengenai alur integrasi aset digital eksternal dari marketplace Fab, penataan material bentang alam multi-layer, sebaran vegetasi lingkungan melalui sistem foliage, serta arsitektur antarmuka pemain modular menggunakan UMG dan Blueprint Interface pada Unreal Engine 5.8. Pemanfaatan paket aset dari platform Fab terbukti mempercepat pembentukan lingkungan visual berkualitas tinggi, di mana mekanisme pemilihan versi kompatibel memungkinkan integrasi paket versi terdahulu ke dalam engine mutakhir tanpa kendala struktural.

Penerapan instans material *MI_landscape_pro_v2_inst* yang didukung kontainer data *Layer Info* berhasil memfasilitasi pengecatan multi-tekstur tanah, rumput, dan genangan air secara fleksibel dengan perhitungan blending bobot yang akurat. Kombinasi metode penempatan vegetasi manual untuk pohon-pohon signifikan dan pemanfaatan *Foliage Mode* untuk sebaran vegetasi massal berbasis instansiasi HISM menghadirkan lingkungan alam yang rimbun namun tetap optimal dalam penggunaan beban pemanggilan penggambaran grafis.

Pembangunan antarmuka pemain berbasis UMG yang dipisahkan secara modular ke dalam *WBP_HUD* dan *WBP_Health* berhasil mewujudkan tata kelola hierarki visual yang rapi. Pemanfaatan pola desain *Blueprint Interface* (*BPI_HUD* dan *BPI_Widget*) yang dipadukan dengan persistensi *Game Instance* mengeliminasi kebutuhan pemanggilan *hard-casting* yang kaku. Ketika karakter menerima kerusakan, data kesehatan diteruskan secara terisolasi untuk memperbarui persentase *HealthBar* dan memicu animasi getar responsif secara seketika, mengonfirmasi seluruh tujuan pembelajaran praktikum telah tercapai dengan sangat baik.

Daftar Pustaka

- [1] Tim Dosen Praktikum Pengembangan Game, “Modul Praktikum Pengembangan Game Pertemuan 7: Asset FAB dan Widget Blueprint (UI)”, Departemen Teknik Elektro dan Informatika, Sekolah Vokasi, Universitas Gadjah Mada, Yogyakarta, 2026.
- [2] Epic Games, “About Fab: The Unified Digital Marketplace for Creators”, 2026. [Daring]. Tersedia: <https://web.archive.org/web/20241022142230/https://www.fab.com/o/about>.
- [3] Epic Games, “Unreal Engine Fab Marketplace Overview”, Epic Developer Community, 2026. [Daring]. Tersedia: <https://dev.epicgames.com/community/learning/talks-and-demos/mJy4/unreal-engine-fab-marketplace-overview>.
- [4] Epic Games, “Landscape Materials in Unreal Engine”, Unreal Engine 5.8 Documentation, 2026. [Daring]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/landscape-materials-in-unreal-engine>.
- [5] Epic Games, “Foliage Mode in Unreal Engine”, Unreal Engine 5.8 Documentation, 2026. [Daring]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/foilage-mode-in-unreal-engine>.
- [6] Epic Games, “UMG UI Designer in Unreal Engine”, Unreal Engine 5.8 Documentation, 2026. [Daring]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-ui-designer-in-unreal-engine>.
- [7] Epic Games, “Animating UMG Widgets in Unreal Engine”, Unreal Engine 5.8 Documentation, 2026. [Daring]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/animating-umg-widgets-in-unreal-engine>.
- [8] Epic Games, “Blueprint Interfaces in Unreal Engine”, Unreal Engine 5.8 Documentation, 2026. [Daring]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-interface-in-unreal-engine>.
- [9] Epic Games, “Game Instance Subsystems in Unreal Engine”, Unreal Engine 5.8 Documentation, 2026. [Daring]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/game-instance-subsystems-in-unreal-engine>.